VISION PAR ORDINATEUR

Reconnaissance de panneaux de signalisation

Du dataset Kaggle à un modèle CNN utilisable dans Google Colab.



Illustration générée avec ChatGPT. Les cadres sont décoratifs ; le notebook classe un panneau déjà recadré.

CLASSIFICATION

43 classes

Catégories de panneaux

DONNÉES

51 839

Images GTSRB

MODÈLE

CNN

TensorFlow / Keras

23 octobre 2026 | Version pédagogique en français

Ce que le projet apprend

Associer une image de panneau à une catégorie connue, à partir d'exemples étiquetés.

Une tâche de classification supervisée

Chaque exemple contient une image RGB et un identifiant de classe compris entre 0 et 42. Le CNN apprend des motifs visuels, puis renvoie 43 scores softmax. La classe au score maximal devient la prédiction.

ENTRÉE

Un panneau cadré

Une image couleur, même si sa taille initiale varie.

APPRENTISSAGE

Image + classe

Le modèle compare sa prédiction à la bonne étiquette.

SORTIE

Une classe parmi 43

Par exemple : limitation à 50 km/h, stop ou travaux.

Le parcours complet du notebook

01 Télécharger – 02 Séparer – 03 Préparer – 04 Entraîner – 05 Évaluer

Classification et détection : deux étapes différentes

Le notebook reçoit un panneau déjà isolé. Pour une photo de rue contenant plusieurs objets, il faut ajouter une étape de détection pour localiser les panneaux, les recadrer, puis classer chaque zone.

Objectif pédagogique
Comprendre un pipeline de vision complet : qualité des données, séparation des ensembles, entraînement, lecture des résultats et réutilisation du modèle.

Le dataset GTSRB

Un benchmark d'images de panneaux allemands, téléchargé depuis Kaggle [1].

TRAIN OFFICIEL

39 209

Avant création de la validation

TEST OFFICIEL

12 630

Réservé à l'évaluation finale

CATÉGORIES

43

Identifiants de 0 à 42

Séparation utilisée par le projet



- **Apprentissage** 31 349 images
- **Validation** 7 860 images
- **Test** 12 630 images

La validation correspond au premier des cinq plis de **StratifiedGroupKFold**. Les vues d'une même séquence restent ensemble : aucune séquence n'est partagée entre apprentissage et validation. La proportion est proche de 80/20 dans le train officiel ; les 43 classes sont présentes des deux côtés.

Exemples réels des données



2 / 50 km/h



14 / Stop



13 / Cédez le passage



17 / Sens interdit



33 / À droite



12 / Route prioritaire



25 / Travaux



18 / Danger

Images ci-dessus : GTSRB, non générées. Les classes sont déséquilibrées et les conditions de prise de vue varient.

Comment fonctionne le CNN

Les convolutions apprennent des motifs utiles pour distinguer les panneaux.



Illustration générée avec ChatGPT : métaphore des pixels, contours et motifs. Ces cartes ne sont pas des activations calculées.

Architecture exacte du notebook

Étape	Opérations principales	Sortie
Entrée	RGB, normalisation et augmentation	48 × 48 × 3
Bloc 1	2 convolutions 3 × 3, 32 filtres ; pooling	24 × 24 × 32
Bloc 2	2 convolutions 3 × 3, 64 filtres ; pooling	12 × 12 × 64
Bloc 3	2 convolutions 3 × 3, 128 filtres ; pooling	6 × 6 × 128
Classification	Flatten ; Dense 128 ; Dense 43 softmax	43 probabilités

Chaque convolution est suivie d’une batch normalization et d’une activation ReLU. Dropout : 0,2 après chaque pooling, puis 0,4 avant la sortie.

Apprendre sans surapprendre

Le meilleur modèle est choisi sur la perte de validation, sans consulter le test officiel.

Préparer les images

Les fichiers PNG sont lus progressivement via **tf.data**, convertis en RGB et redimensionnés à 48 × 48 avec interpolation bilinéaire et antialiasing. Le modèle applique ensuite la normalisation de 0-255 vers 0-1.

Paramètre	Valeur dans le notebook
Batch et durée maximale	64 images ; 20 époques au maximum
Optimiseur et perte	Adam, taux initial 0,001 ; entropie croisée sparse
Augmentation	Rotation ±10,8° ; translation jusqu'à 8 % ; zoom léger de 10 %
Retournements	Aucun : ils peuvent changer le sens d'un panneau
Arrêt anticipé	Patience de 5 époques sur val_loss
Réduction du taux	Facteur 0,5 après 2 époques de stagnation ; minimum 0,000001
Reproductibilité	Graine 42 ; version du dataset fixée à 1

Une époque, étape par étape

01 Prédire les classes pour un batch d'images augmentées.

02 Calculer l'erreur et ajuster les poids par rétropropagation.

03 Mesurer la perte sur la validation, sans augmentation aléatoire.

04 Sauvegarder le checkpoint si cette perte s'améliore.

Le GPU accélère le calcul. La graine aide à reproduire l'expérience, mais des différences matérielles ou logicielles peuvent encore modifier les résultats.

Mesurer ce que le modèle sait

Recharger le meilleur checkpoint, puis prédire les images du test dans leur ordre d'origine.

Accuracy

Part des images correctement classées.

Prédictions correctes / nombre total d'images.

F1 macro

Moyenne du F1 de chacune des 43 classes.

Chaque classe a le même poids, même lorsqu'elle est rare.

Matrice de confusion

Vraies classes en lignes, classes prédites en colonnes.

La normalisation par ligne facilite la lecture du rappel par classe.

Ce qui a déjà été vérifié

Une exécution technique courte a utilisé **344 images d'apprentissage, 172 de validation et 172 de test**, sur une seule époque, avec TensorFlow CPU 2.21 et Python 3.12.

- Schéma Jupyter et syntaxe Python valides ; lecture de vraies images GTSRB.
- Entraînement, évaluation et prédiction exécutés jusqu'au bout.
- Export du modèle, rechargement avec comparaison des sorties et archive ZIP vérifiés.

Performances finales : à mesurer
Le test court valide le fonctionnement du pipeline. Il ne mesure pas la qualité finale du classifieur.
L'entraînement complet reste à lancer dans Colab ; aucun score de performance final n'est revendiqué ici.

Utiliser le projet dans Colab

Le notebook contient déjà les cellules nécessaires, du téléchargement à la prédiction.

1. **Ouvrir le notebook**

   Utiliser le lien Colab du dépôt GitHub, ou importer traffic_sign_recognition.ipynb via Fichier > Importer un notebook.

2. **Choisir le mode d'exécution**

   Sélectionner un GPU si disponible. Garder SMOKE_TEST = False pour entraîner sur les ensembles complets.

3. **Télécharger les données**

   Exécuter les cellules dans l'ordre. KaggleHub télécharge la version 1. Si nécessaire, utiliser kagglehub.login() séparément ; ne pas publier de clé.

4. **Entraîner et lire les résultats**

   Examiner les courbes et la validation, puis le rapport du test. Éviter de régler les paramètres à répétition à partir du score de test.

5. **Essayer une image personnelle**

   Activer RUN_UPLOAD_DEMO = True puis relancer la cellule de prédiction. Recadrer l'image sur un seul panneau appartenant aux 43 classes.

6. **Conserver le travail**

   Activer DOWNLOAD_RESULTS = True pour récupérer l'archive avant la fin de la session. Les fichiers temporaires Colab peuvent disparaître.

Ce que contient l'archive

traffic_sign_model.keras, class_names.json, metrics.json, metadata.json, splits.csv, courbes d'apprentissage, rapport par classe, matrice de confusion et prédictions du test.

Ouvrir le notebook dans Google Colab

Interpréter avec prudence

Un projet pédagogique pour apprendre une méthode complète de classification d'images.

- **Une catégorie connue est toujours choisie.** Un objet inconnu ou une image sans panneau peut recevoir une prédiction très confiante.
- **Le contexte peut changer.** Des panneaux d'autres pays, le flou, l'éclairage ou des occultations peuvent dégrader la reconnaissance.
- **Un score softmax n'est pas une garantie.** Il faut une validation supplémentaire pour tout usage réel ; ce projet ne sert pas à prendre des décisions de conduite.

Pistes d'amélioration
Analyser les erreurs par classe, comparer des tailles d'image sur la validation, tester d'autres architectures et ajouter un détecteur pour traiter une scène complète.

Références et accès au projet

[1] Kaggle - GTSRB, version 1
Source du dataset et de ses conditions d'utilisation.

[2] Dépôt GitHub - notebook et README
Implémentation et paramètres décrits dans ce document.

[3] KaggleHub - documentation officielle
Téléchargement et authentification.

[4] TensorFlow - augmentation d'images
Principes des couches d'augmentation.

[5] Scikit-learn - StratifiedGroupKFold
Séparation stratifiée respectant les groupes.

Crédits visuels. Deux illustrations créées avec l'outil intégré de génération d'images de ChatGPT. Elles ne représentent pas des résultats mesurés. Les huit vignettes de panneaux proviennent du dataset GTSRB. Prompts et images conservés dans le dossier assets du rapport.

Document préparé à partir du notebook du projet et du journal de vérification locale. Code sous licence MIT ; données soumises aux conditions de leur source. État du projet au 23 octobre 2026.